Divide and Conquer

Divide and Conquer

The **divide-and-conquer** paradigm (3 steps),

Divide and Conquer

The **divide-and-conquer** paradigm (3 steps),

- **Divide**
- **Conquer**
- **Combine**

Divide and Conquer

The **divide-and-conquer** paradigm (3 steps),

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Divide and Conquer

The **divide-and-conquer** paradigm (3 steps),

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Divide and Conquer

The **divide-and-conquer** paradigm,

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

How long can you recurse?

Divide and Conquer

The **divide-and-conquer** paradigm,

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the subproblems are small enough, just solve them by brute force.

How to obtain a solution to the original problem?

Divide and Conquer

The **divide-and-conquer** paradigm,

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the subproblems are small enough, just solve them by brute force.

Combine the subproblem solutions to form a solution to the original problem.

Divide and Conquer

Use a **recurrence** to characterize the running time of a divide-and-conquer algorithm. Solving the recurrence gives us the asymptotic running time.

A ***recurrence*** is a function is defined in terms of

- one or more base cases, and
- itself, with smaller arguments.

Question

What is the recurrence relation for the following algorithm:

we divide the problem into

- “a” subproblems,
- each of size “1/b” of the original instance, and
- the divide and combine steps takes f(n) time.

Question

What is the recurrence relation for the following algorithm:

we divide the problem into

- “a” subproblems,
- each of size “1/b” of the original instance, and
- the divide and combine steps takes f(n) time.

$$T(n)=aT(n/b)+f(n)$$

Have you seen any D&C algorithms?

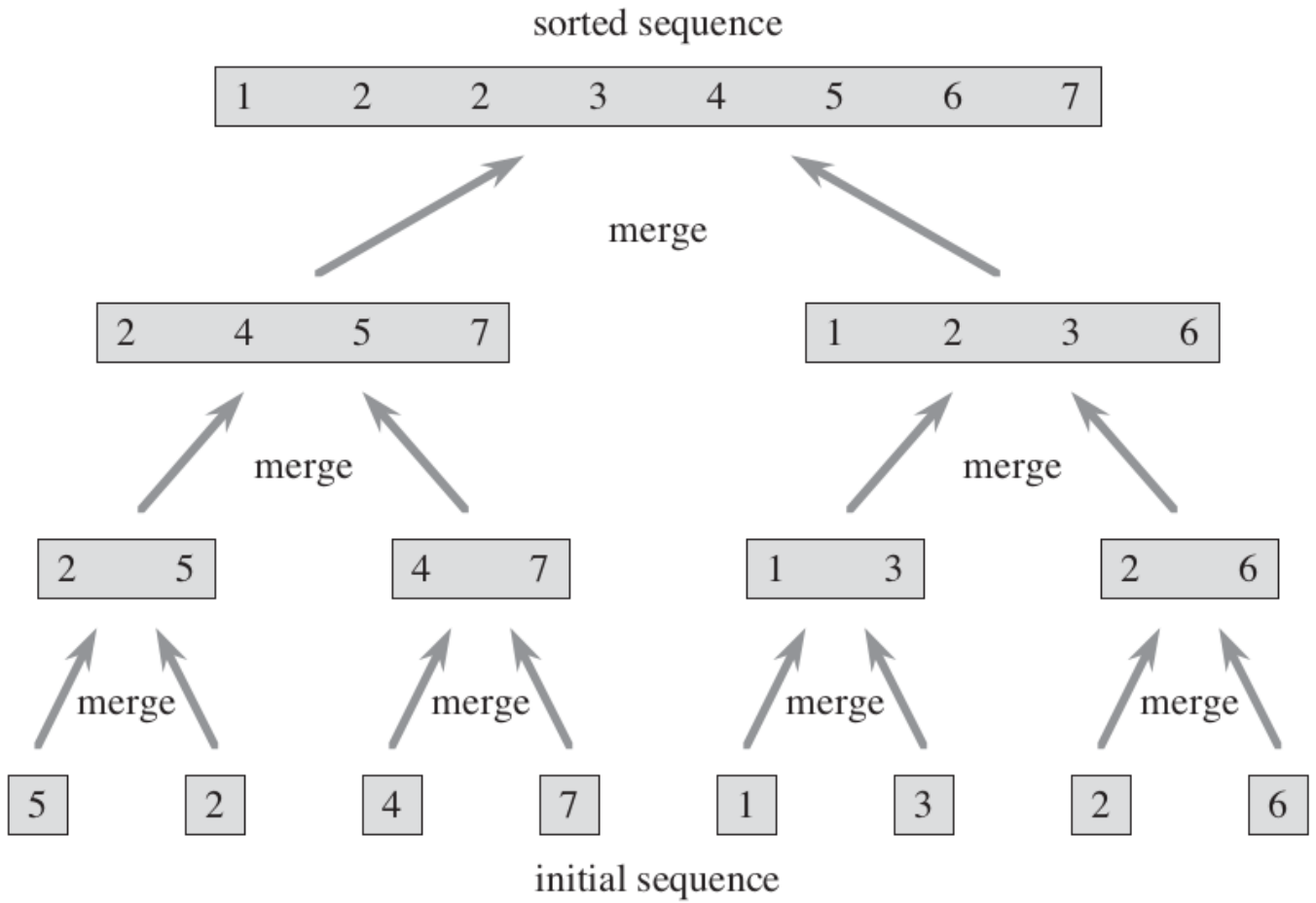
Sorting

- Divide
- Conquer
- Combine

- **Input:** 5 2 4 7 1 3 2 6

Sorting

- Divide
- Conquer
- Combine



Algorithm

Algorithm

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Algorithm

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

```
MERGE(A, p, q, r)
 1  n1 = q − p + 1
 2  n2 = r − q
 3  let L[1 .. n1 + 1] and R[1 .. n2 + 1] be new arrays
 4  for i = 1 to n1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n2
 7      R[j] = A[q + j]
 8  L[n1 + 1] = ∞
 9  R[n2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

Algorithm

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

How much time would MERGE procedure take?

```
MERGE(A, p, q, r)
 1  n1 = q − p + 1
 2  n2 = r − q
 3  let L[1 .. n1 + 1] and R[1 .. n2 + 1] be new arrays
 4  for i = 1 to n1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n2
 7      R[j] = A[q + j]
 8  L[n1 + 1] = ∞
 9  R[n2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

Divide:

Conquer:

Combine:

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. **$D(n)=\Theta(1)$**.

Conquer:

Combine:

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. **$D(n)=\Theta(1)$**.

Conquer: We recursively solve two subproblems, each of size n/2,

Combine:

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. **$D(n)=\Theta(1)$**.

Conquer: We recursively solve two subproblems, each of size n/2, which contributes **$2T(n/2)$** to the running time.

Combine:

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. **$D(n)=\Theta(1)$**.

Conquer: We recursively solve two subproblems, each of size n/2, which contributes **$2T(n/2)$** to the running time.

Combine: We have already noted that the MERGE procedure on an n-element subarray takes time $\Theta(n)$. **$C(n)=\Theta(n)$**.

Recurrence

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 , \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 . \end{cases}$$

Correctness?

At the start of each iteration of the **for** loop of lines 12–17, the subarray $A[p \,..\, k-1]$ contains the $k-p$ smallest elements of $L[1 \,..\, n_1+1]$ and $R[1 \,..\, n_2+1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A.

Question

Let T(n) denote the worst-case running time of MERGE-SORT on n numbers.

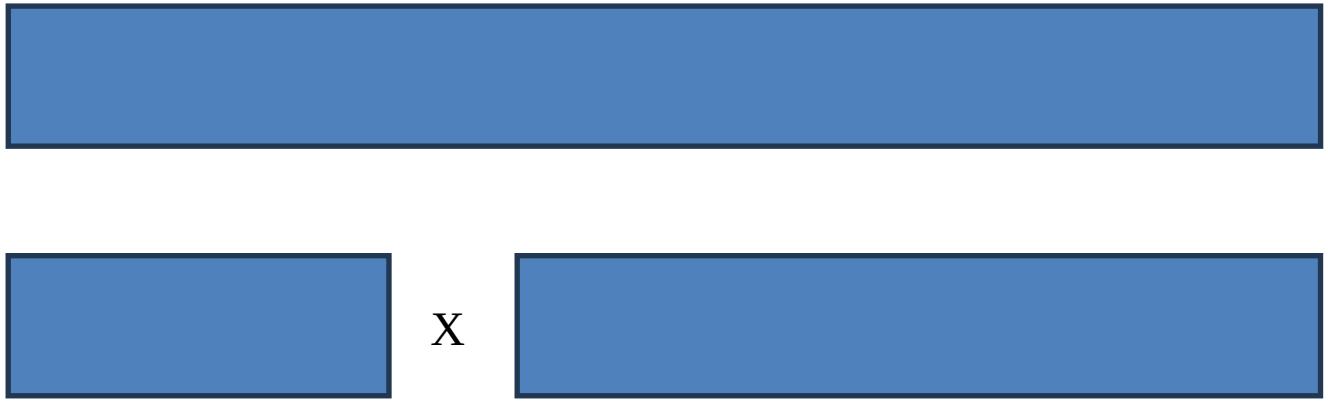
Divide: The divide step just computes the middle of the subarray, which takes constant time. **$D(n)=\Theta(1)$**.

Conquer: We recursively solve two subproblems, each of size n/2, which contributes **$2T(n/2)$** to the running time.

Combine: We have already noted that the MERGE procedure on an n-element subarray takes time $\Theta(n)$. **$C(n)=\Theta(n)$**.

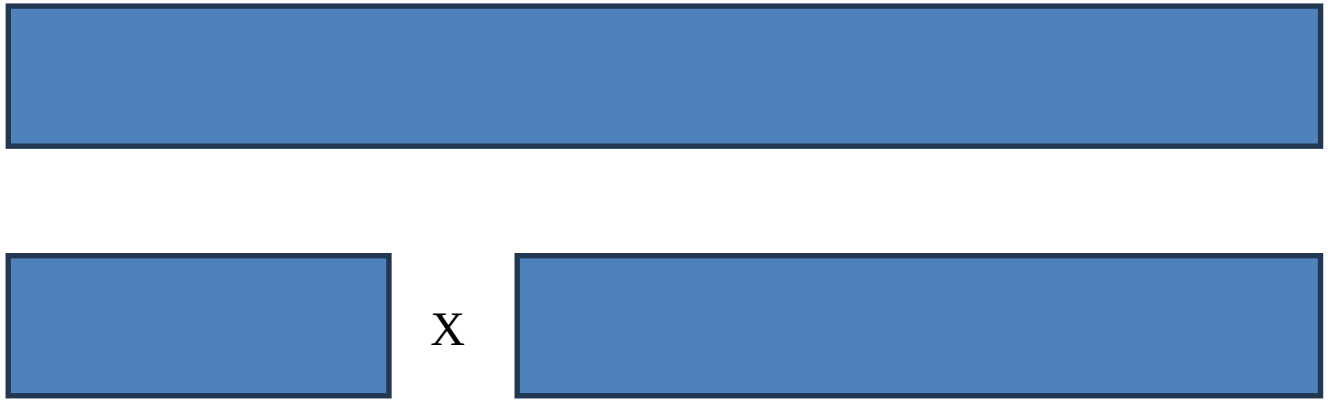
Design another strategy that does not take much in the combine step!

Sorting



- Dividing the problem into two subproblems
  - One that contains values less than X
  - The other containing values greater than or equal to X.

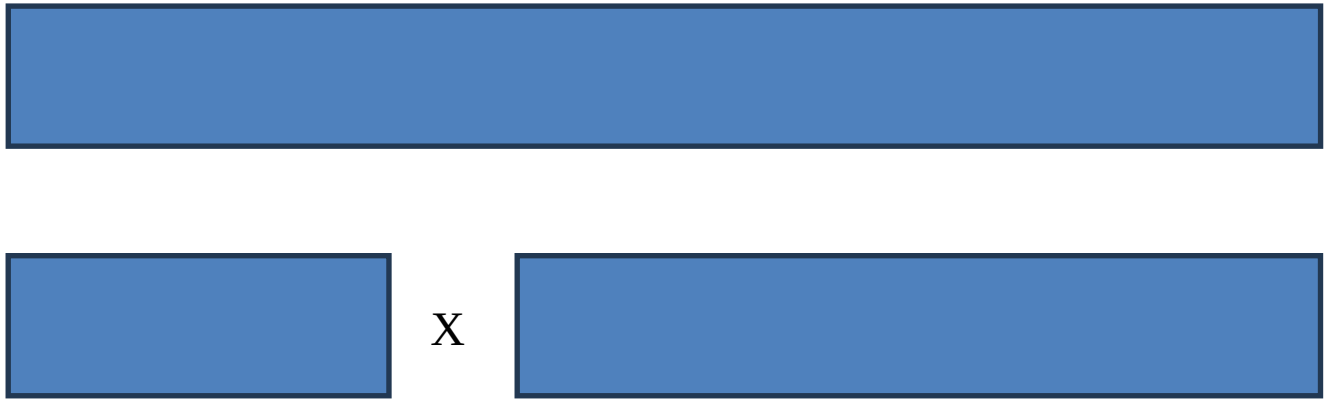
Sorting



- Dividing the problem into two subproblems
  - One that contains values less than X
  - The other containing values greater than or equal to X.

Can you model it as a D&C algorithm?

Sorting



- Dividing the problem into two subproblems
  - One that contains values less than X
  - The other containing values greater than or equal to X.

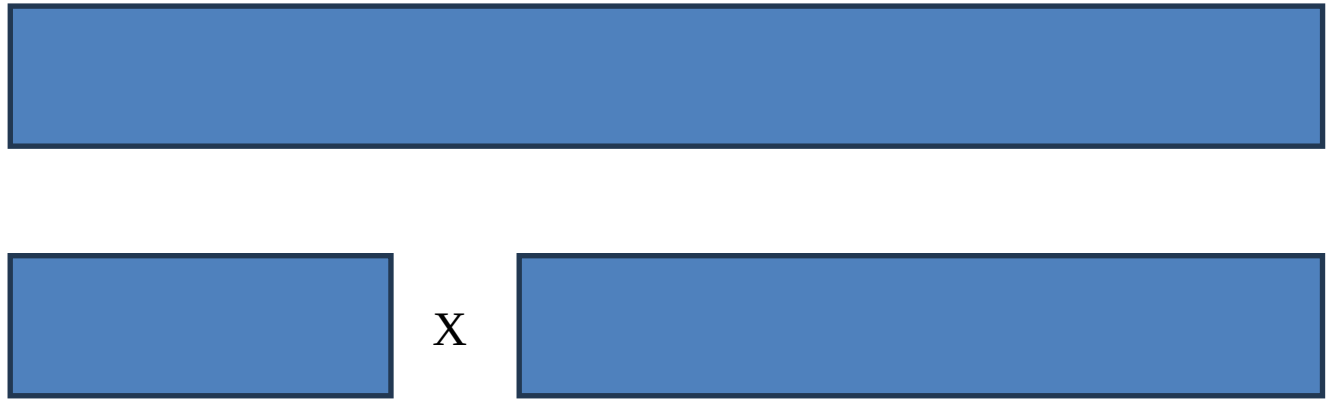
How much time does the combine step take?

Sorting – strategy 2

- Dividing the problem into two subproblems
- Recursively divide it until you hit the base case.
- Merge/combine two sorted sub-arrays into a sorted array. **O(1)**

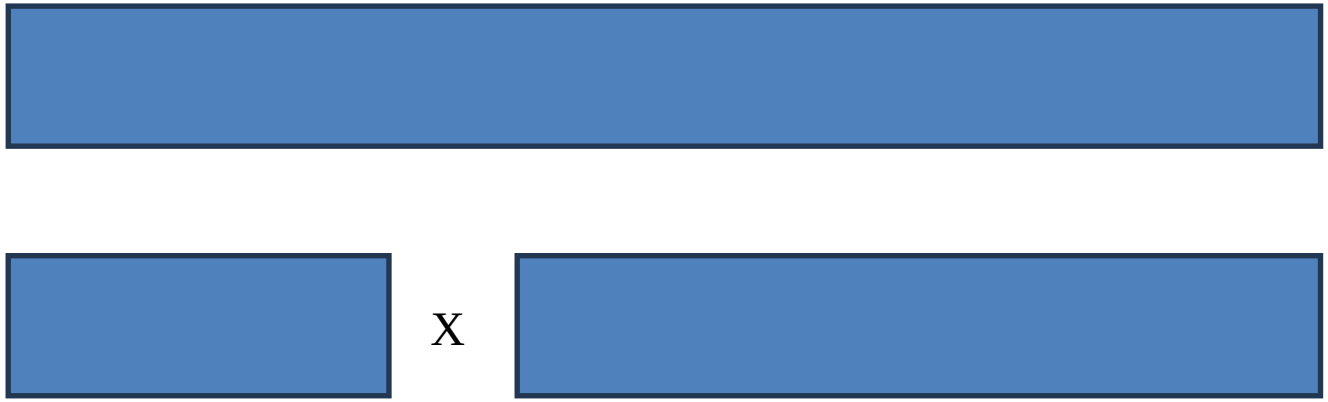
Do you recall this algorithm?

Quick Sort



- Choose a pivot and separate the sub-arrays in such a way that
  - One that contains values less than X
  - The other containing values greater than or equal to X.

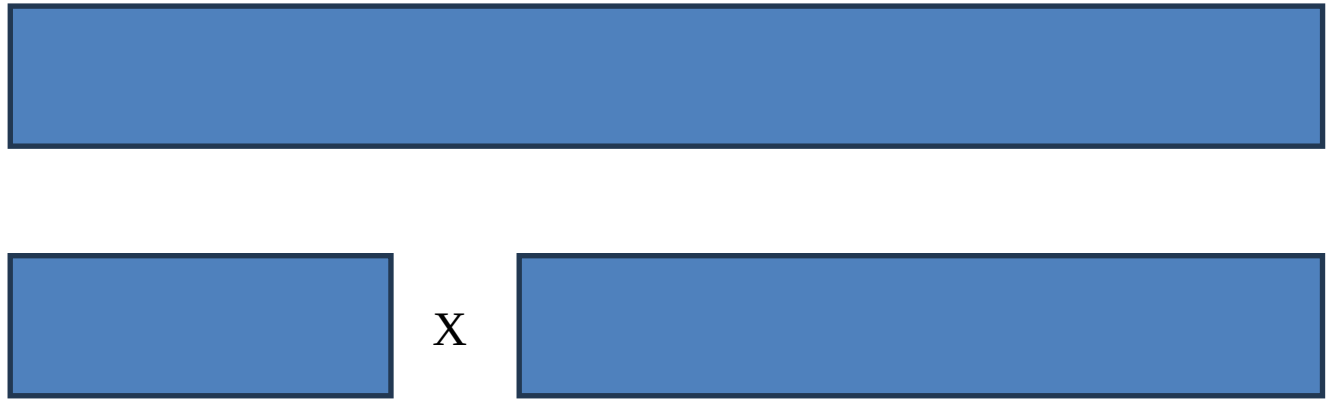
Quick Sort



- Choose a pivot and separate the sub-arrays in such a way that
  - One that contains values less than X
  - The other containing values greater than or equal to X.
- Recurse on the sub-arrays
- Put them together

Quick Sort

How does it differ from Merge sort?



- Choose a pivot and separate the sub-arrays in such a way that
  - One that contains values less than X
  - The other containing values greater than or equal to X.
- Recurse on the sub-arrays
- Put them together

Quick Sort - time

- The running time depends on which pivot element you choose.
- **Rank** of an element is its position in the sorted order (assume distinct elements).
- Let T(n) be the running time of the algorithm. Then

Quick Sort - time

- The running time depends on which pivot element you choose.
- **Rank** of an element is its position in the sorted order (assume distinct elements).
- Let T(n) be the running time of the algorithm. Then

T(n) = Time taken to divide the problem into sub-problems
+ Time taken to solve the sub-problems
+ Time taken to combine the solutions.

- If the rank of the pivot is i, then

Quick Sort - time

- The running time depends on which pivot element you choose.
- **Rank** of an element is its position in the sorted order (assume distinct elements).
- Let T(n) be the running time of the algorithm. Then

T(n) = Time taken to divide the problem into sub-problems
+ Time taken to solve the sub-problems
+ Time taken to combine the solutions.

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Quick Sort running time

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element
- Middle element
- Greatest element

Quick Sort running time

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element
- Middle element
- Greatest element

$$T(n) = T(n-1) + n-1 = O(n^2)$$

Quick Sort running time

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element — $T(n) = T(n-1) + n-1 = O(n^2)$
- Middle element — $T(n) = 2T(n/2) + n-1 = O(n \log n)$
- Greatest element — $T(n) = T(n-1) + n-1 = O(n^2)$

Quick Sort running time

What is the running time of Quick sort (this implementation)? = **$O(n^2)$**

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element — $T(n) = T(n-1) + n-1 = O(n^2)$
- Middle element — $T(n) = 2T(n/2) + n-1 = O(n \log n)$
- Greatest element — $T(n) = T(n-1) + n-1 = O(n^2)$

Quick Sort running time

Can you design an efficient algorithm?

- If the rank of the pivot is i, then

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element — $T(n) = T(n-1) + n-1 = O(n^2)$
- Middle element — $T(n) = 2T(n/2) + n-1 = O(n \log n)$
- Greatest element — $T(n) = T(n-1) + n-1 = O(n^2)$

Quick Sort running time

- If the rank of the pivot is i, then

Can you design an efficient algorithm?

- **Yes, if I could find the middle element each time.**

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

Question. What happens if each time you choose as pivot the

- Minimum element — $T(n) = T(n-1) + n-1 = O(n^2)$
- Middle element — $T(n) = 2T(n/2) + n-1 = O(n \log n)$
- Greatest element — $T(n) = T(n-1) + n-1 = O(n^2)$

Quick Sort running time

- If the rank of the pivot is i, then

Can you design an efficient algorithm?

- **Yes, if I could find the middle element each time "efficiently" ~ O(n).**

$$T(n) = T(i-1) + T(n-i) + (n-1)$$

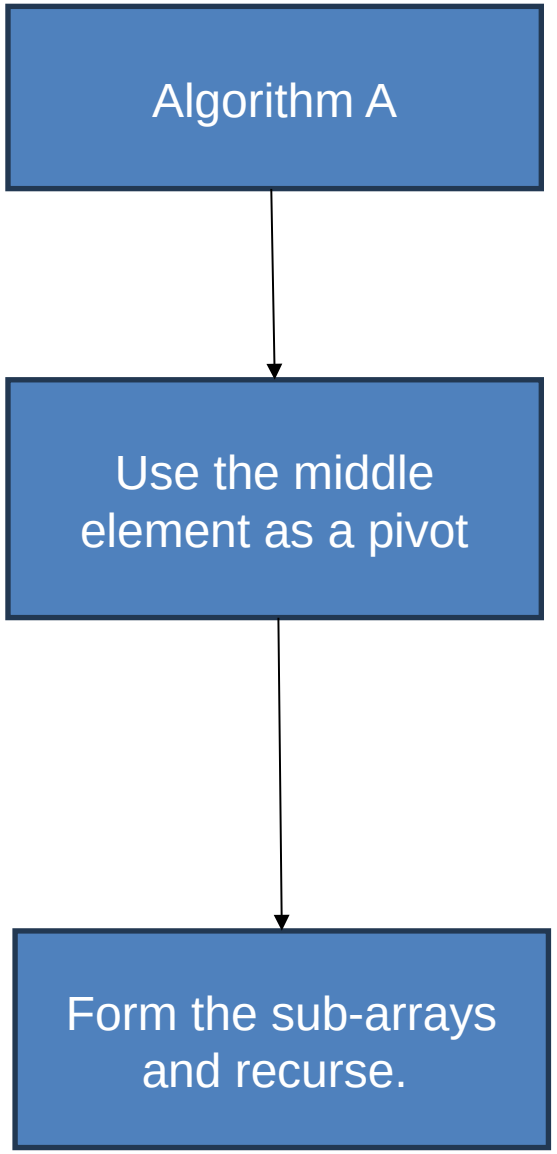
Question. What happens if each time you choose as pivot the

- Minimum element — $T(n) = T(n-1) + n-1 = O(n^2)$
- Middle element — $T(n) = 2T(n/2) + n-1 = O(n \log n)$
- Greatest element — $T(n) = T(n-1) + n-1 = O(n^2)$

Goal

- Input: Given an array of **n** numbers.
- Output: the middle element in O(n) time.
  (element of rank n/2 or median)

Algorithm A: for the above problem.



Median finding

Any algorithm?

- Input: Given an array of **n** numbers.
- Output: the middle element in O(n) time.
  (element of rank n/2 or median)

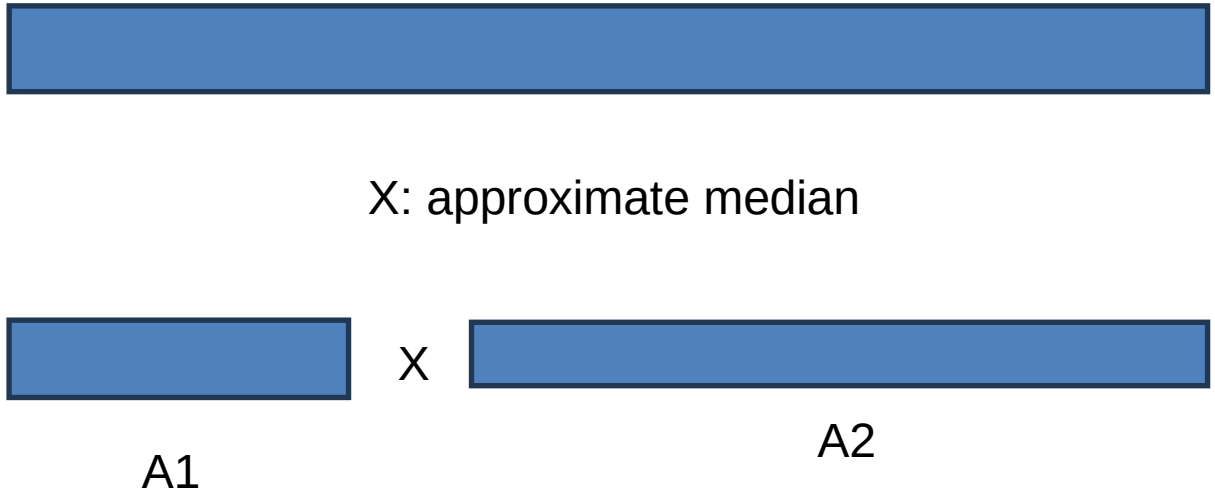
Median finding

Any algorithm?

- Input: Given an array of **n** numbers.
- Output: the middle element in O(n) time.
  (element of rank n/2 or median)

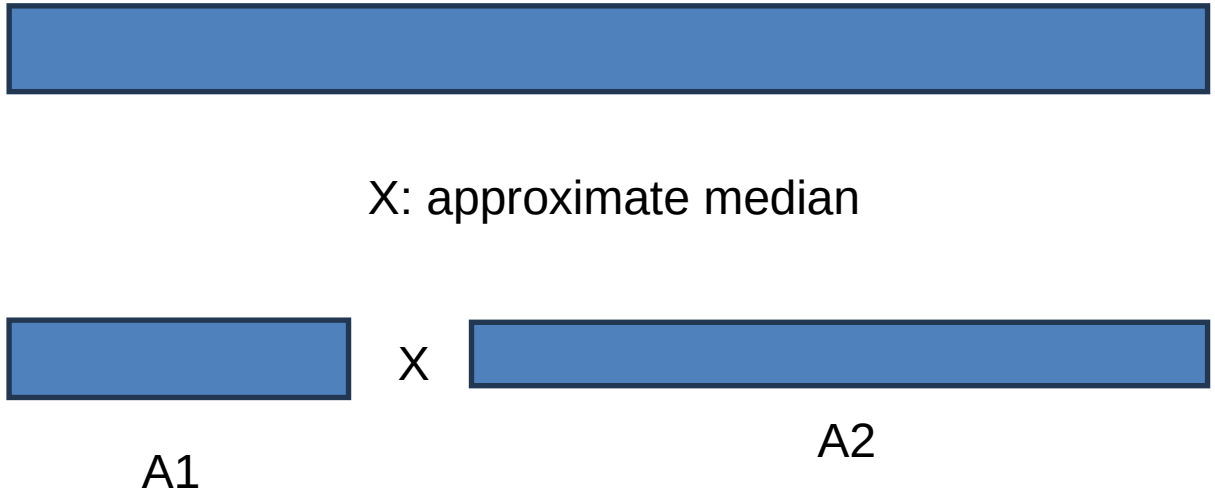
Use D&C strategy to find an algorithm.

Median finding using Approximate median



- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then

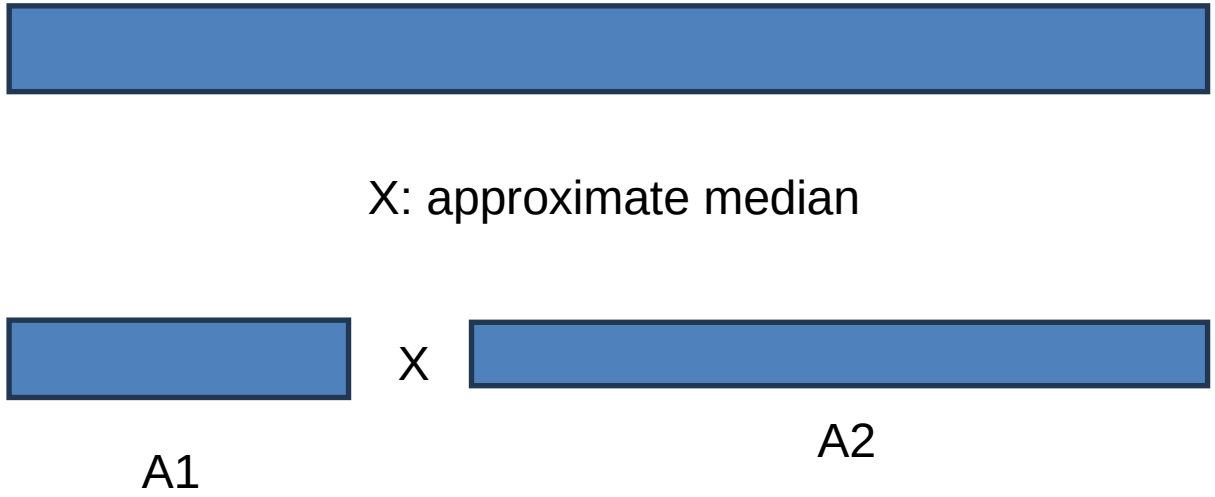
Median finding using Approximate median



- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**

Median finding using Approximate median

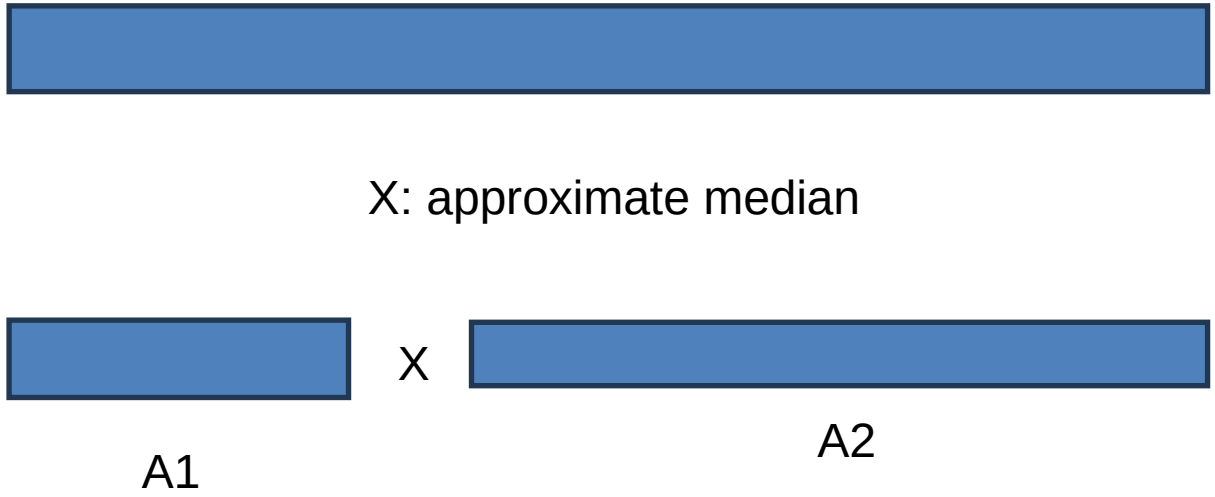
How much time does computing median take?



- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**
- Suppose that approximate median can be found in linear time.

Median finding using Approximate median

How much time does computing median take?

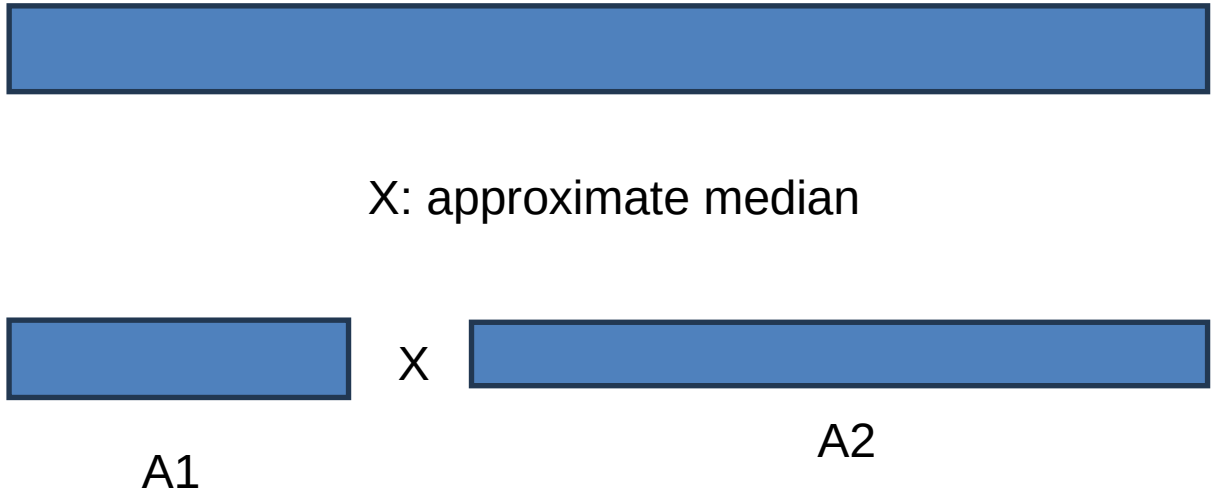


T(n) = c.n + T(3n/4)

- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**
- Suppose that approximate median can be found in linear time.

Median finding using Approximate median

How much time does computing median take?



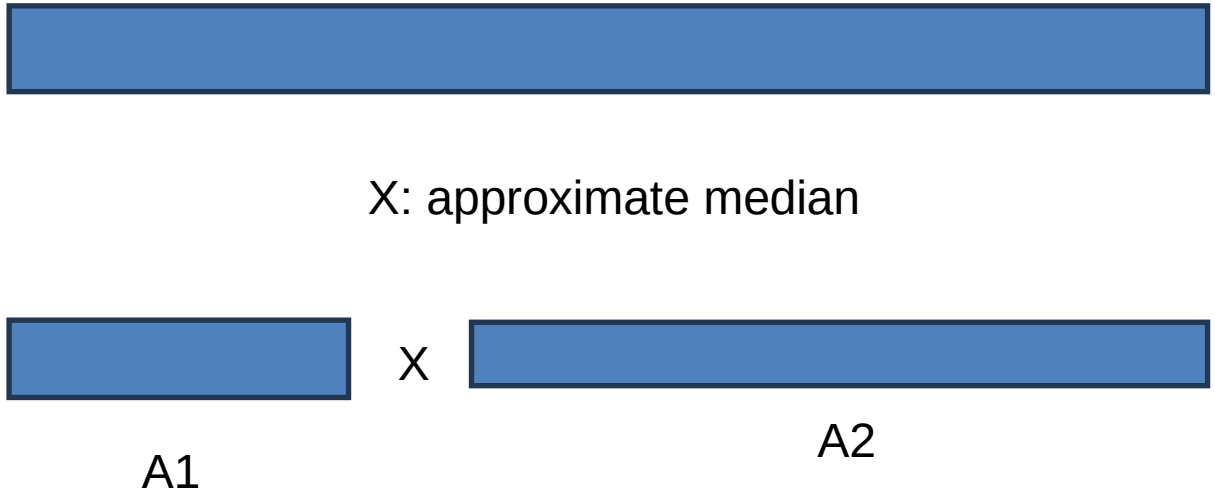
T(n) = c.n + T(3n/4)

O(n)

- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**
- Suppose that approximate median can be found in linear time.

Median finding using Approximate median

Are we done?

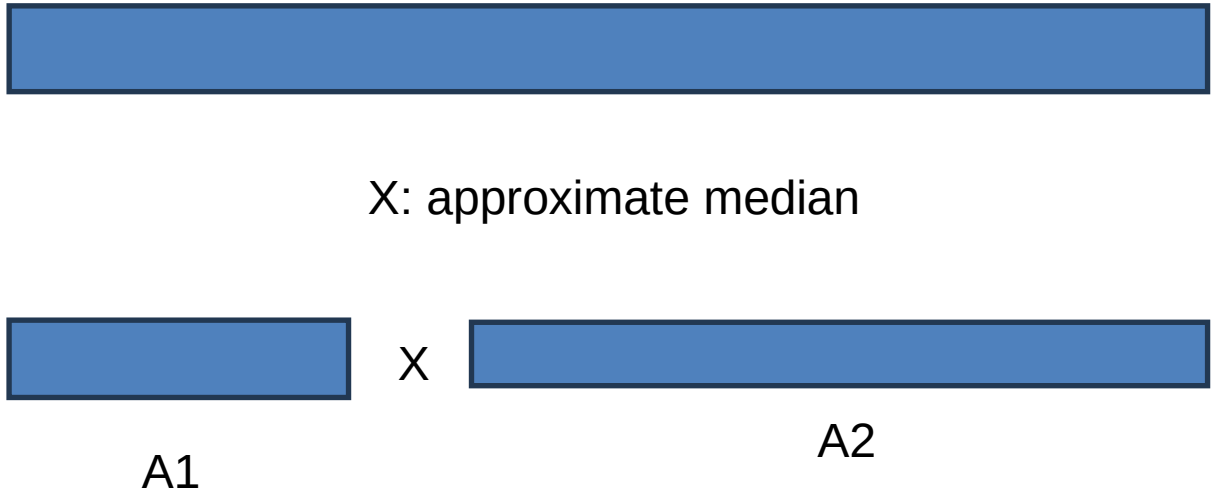


T(n) = c.n + T(3n/4)

O(n)

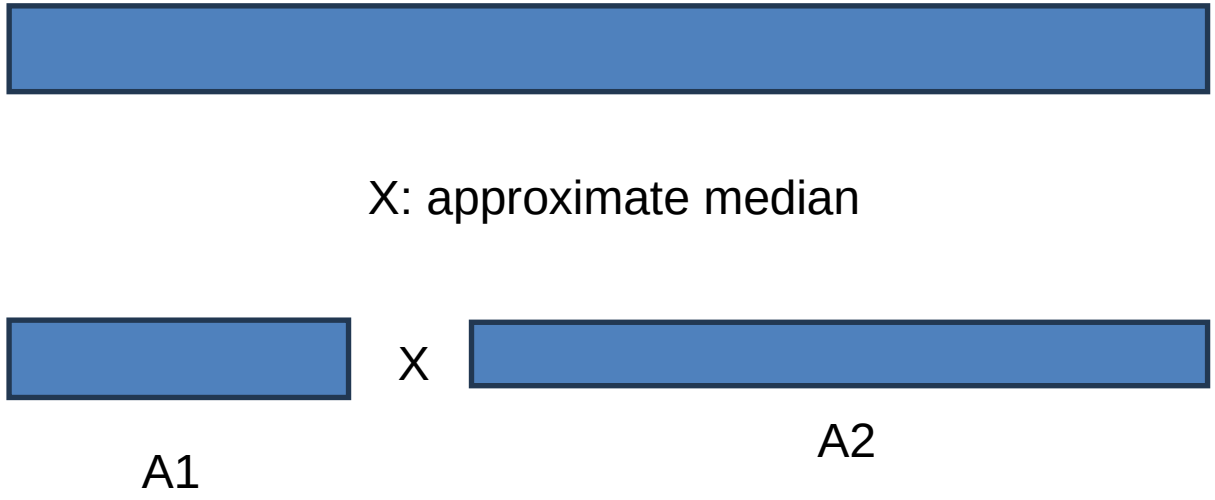
- If the rank of X is n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**
- Suppose that approximate median can be found in linear time.

Median finding using Approximate median



- The median of A may not be the median of A2.
- It depends on |A1|. What is it exactly?

Median finding using Approximate median

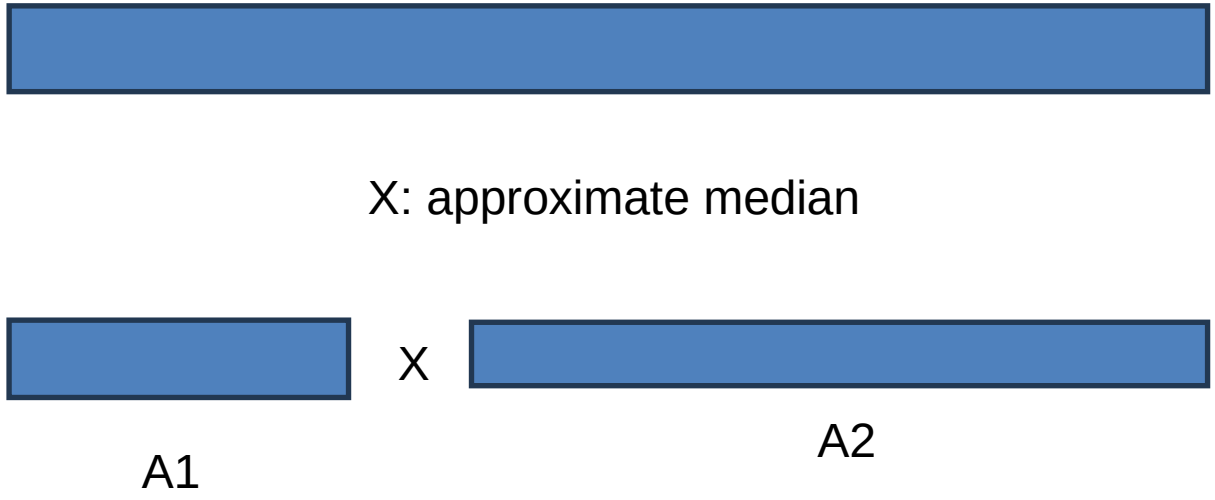


- The median of A may not be the median of A2.
- It depends on |A1|. What is it exactly?

The median of A is the element of rank **n/2 – (|A1|+1)** in **A2**.

Median finding using Approximate median

What do we do?

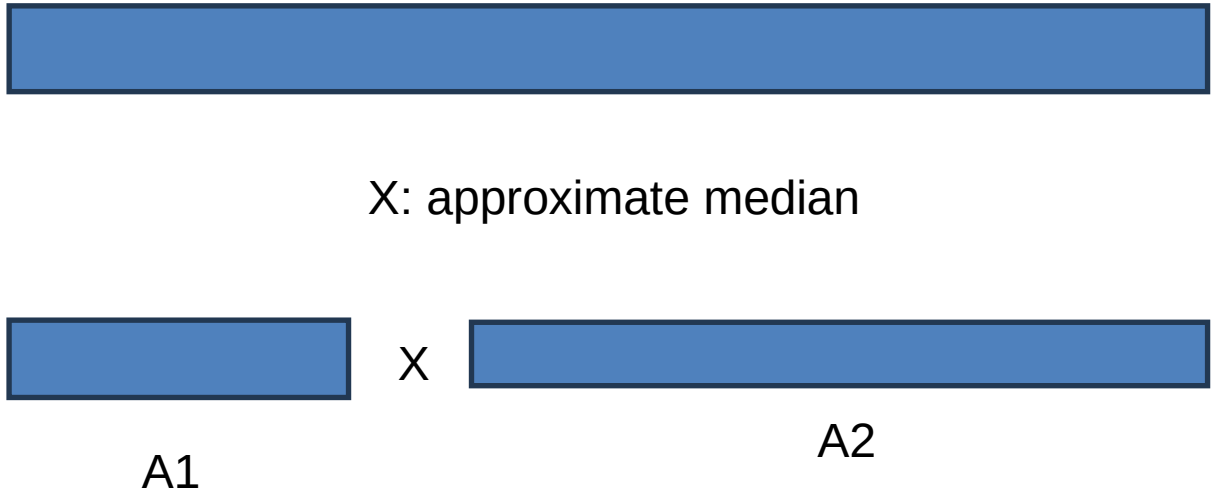


- The median of A may not be the median of A2.
- It depends on |A1|. What is it exactly?

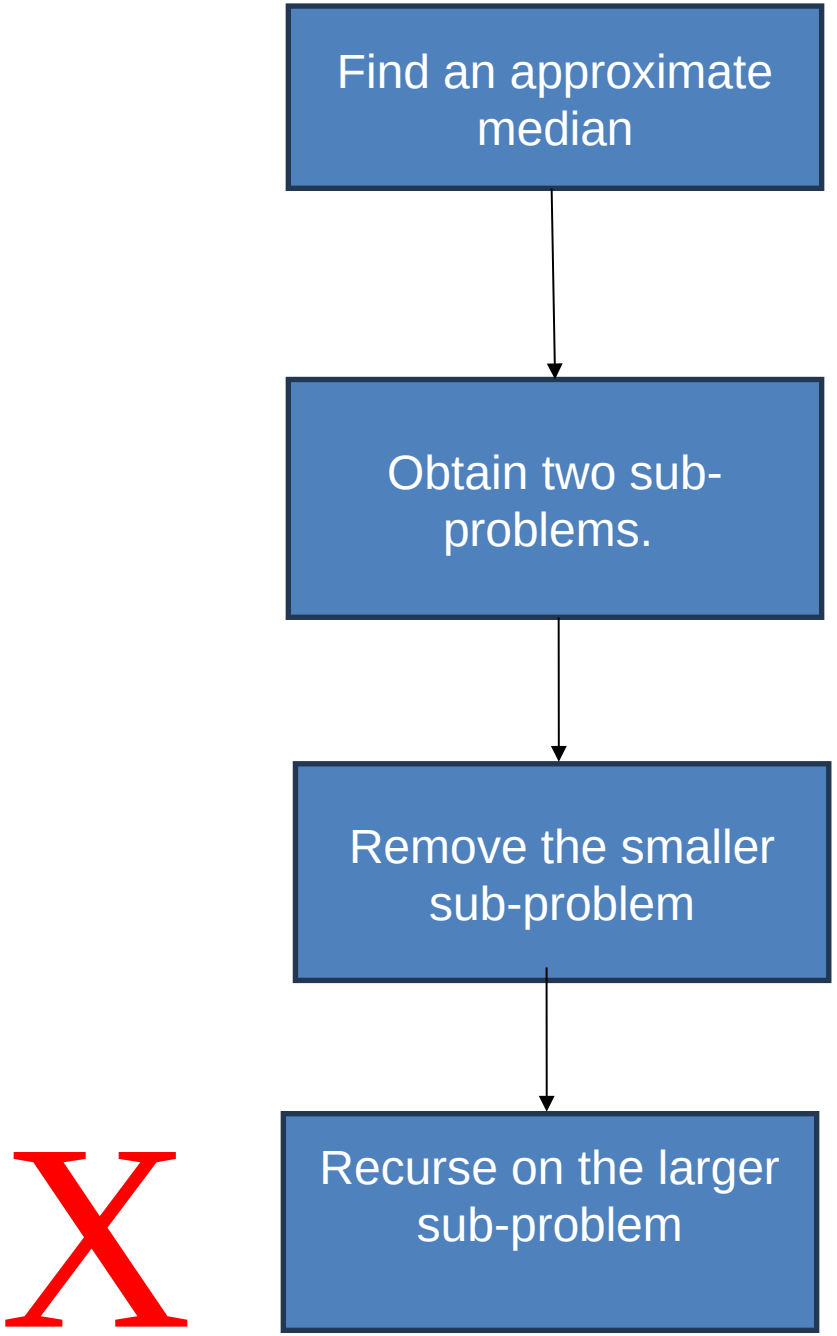
The median of A is the element of rank **n/2 – (|A1|+1)** in **A2**.

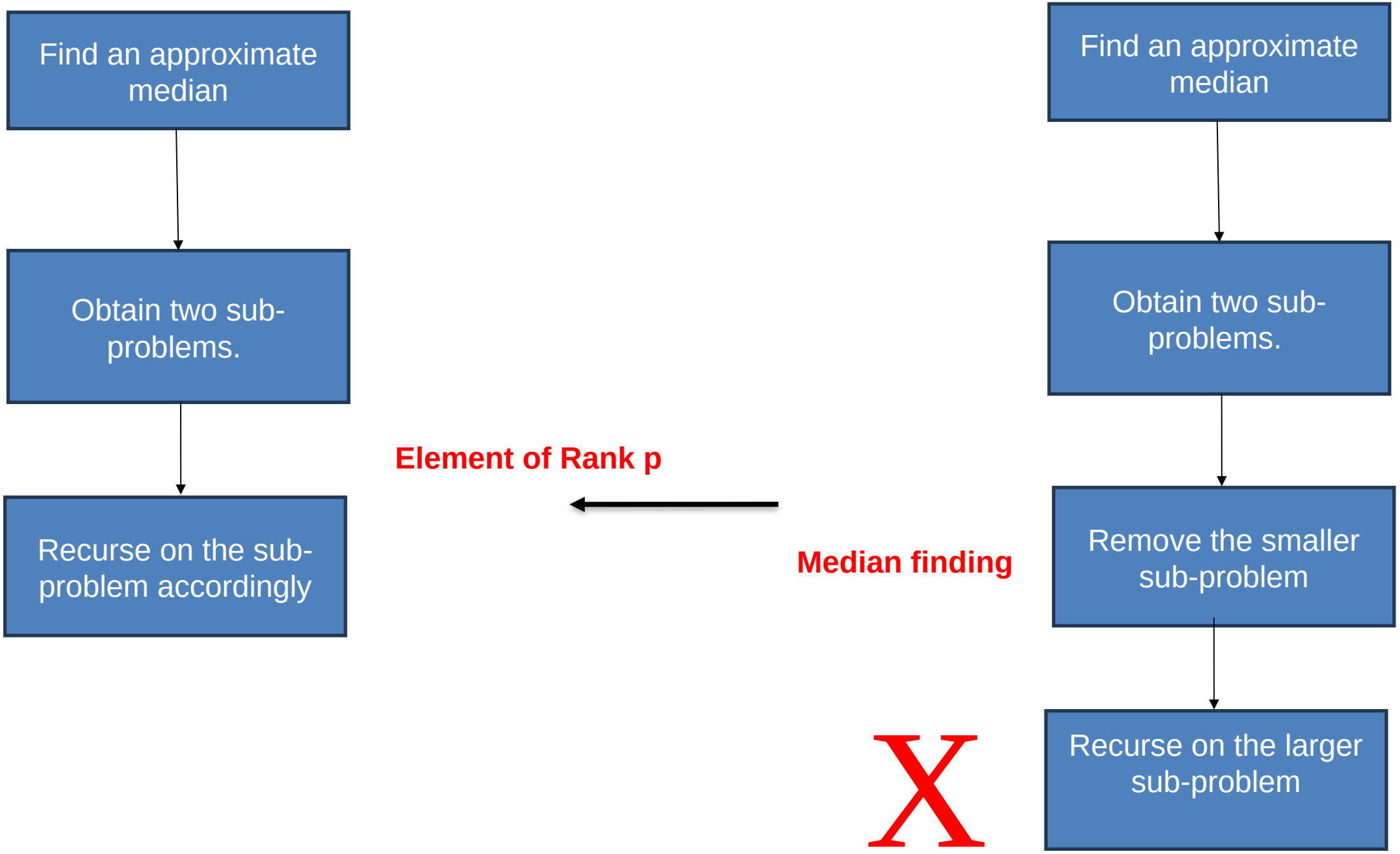
Median finding using Approximate median

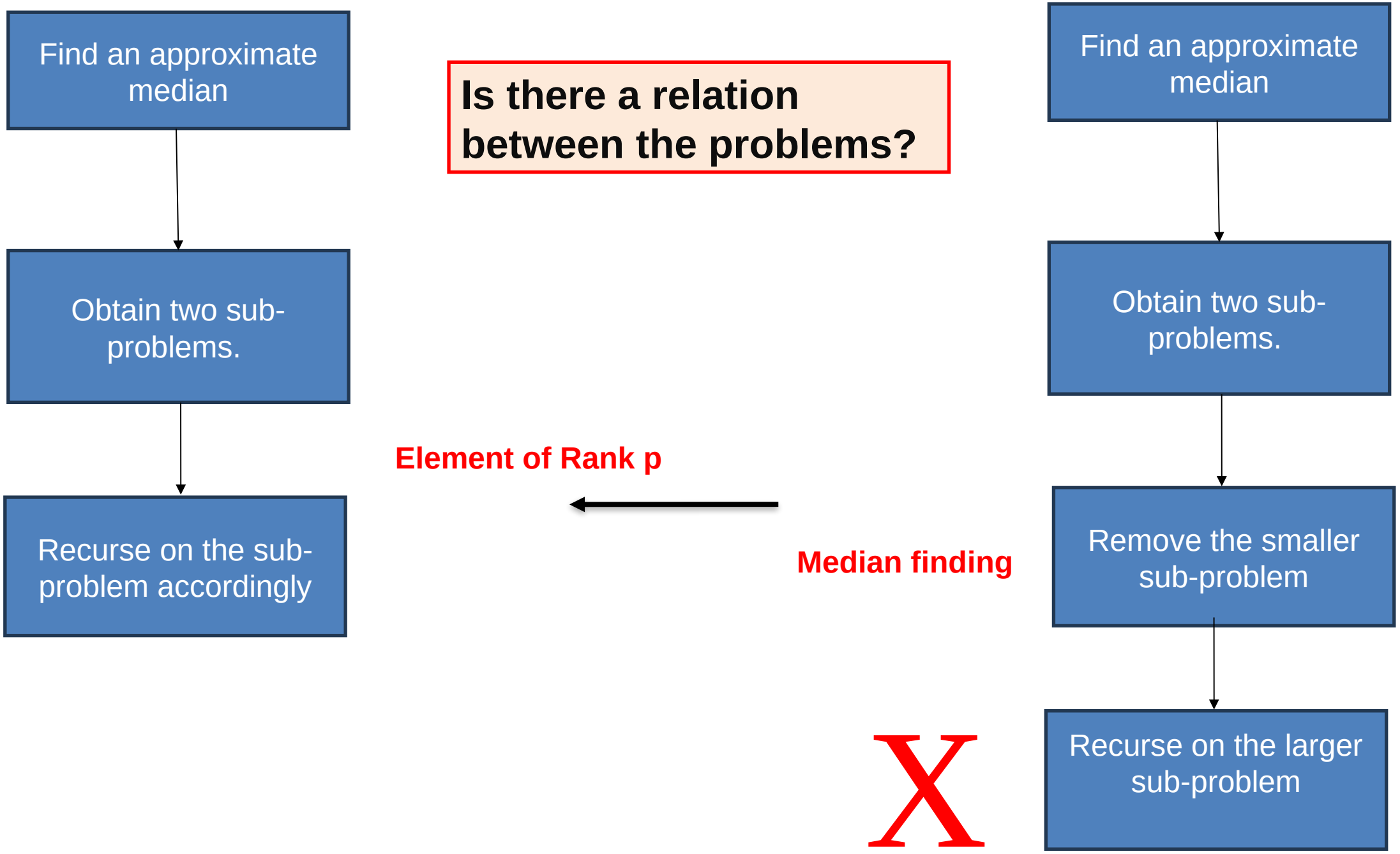
Change the problem!



- The median of A is the element of rank **n/2 – (|A1|+1)** in **A2**.
- Instead of looking at the median of A, we will **look for the element of rank n/2 in A**.
- Therefore in A2 we could look for the element of rank **n/2 – (|A1|+1).**


Find an approximate median
Obtain two sub-problems.
Remove the smaller sub-problem
Recurse on the larger sub-problem
X

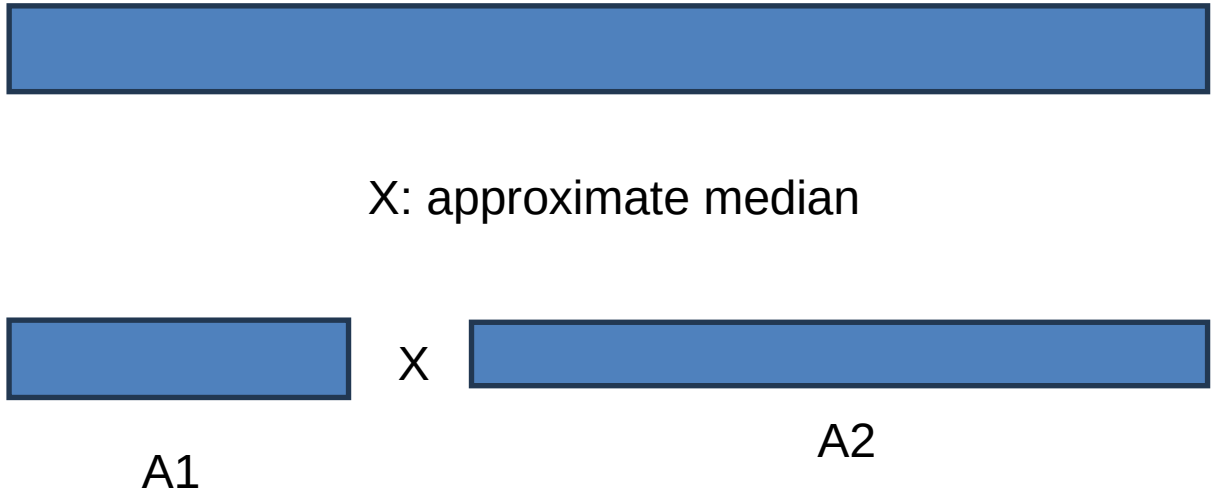

Find an approximate median
Obtain two sub-problems.
Recurse on the sub-problem accordingly
Element of Rank p
Median finding
Find an approximate median
Obtain two sub-problems.
Remove the smaller sub-problem
Recurse on the larger sub-problem
X

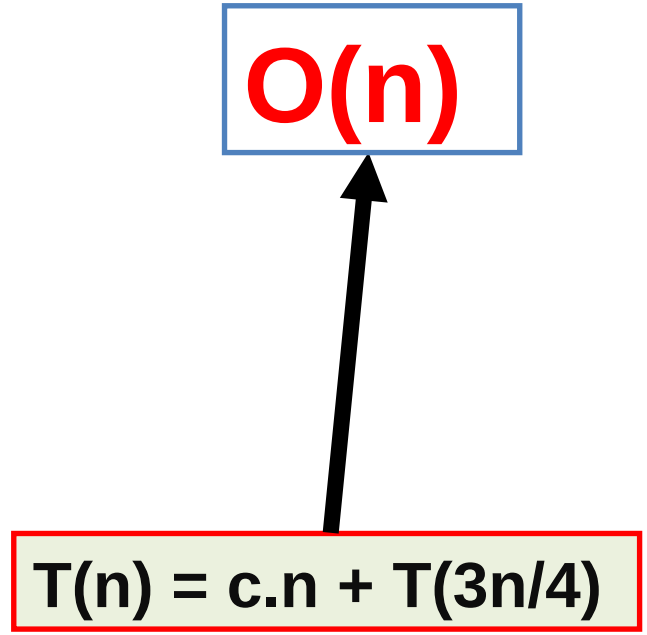

Find an approximate median
Obtain two sub-problems.
Recurse on the sub-problem accordingly
Is there a relation between the problems?
Element of Rank p
Median finding
X
Find an approximate median
Obtain two sub-problems.
Remove the smaller sub-problem
Recurse on the larger sub-problem

Element of rank p

- **Input**: Given an array of **n** numbers, and an integer p.
- **Output**: the element of rank p.

Element of Rank p using Approximate median

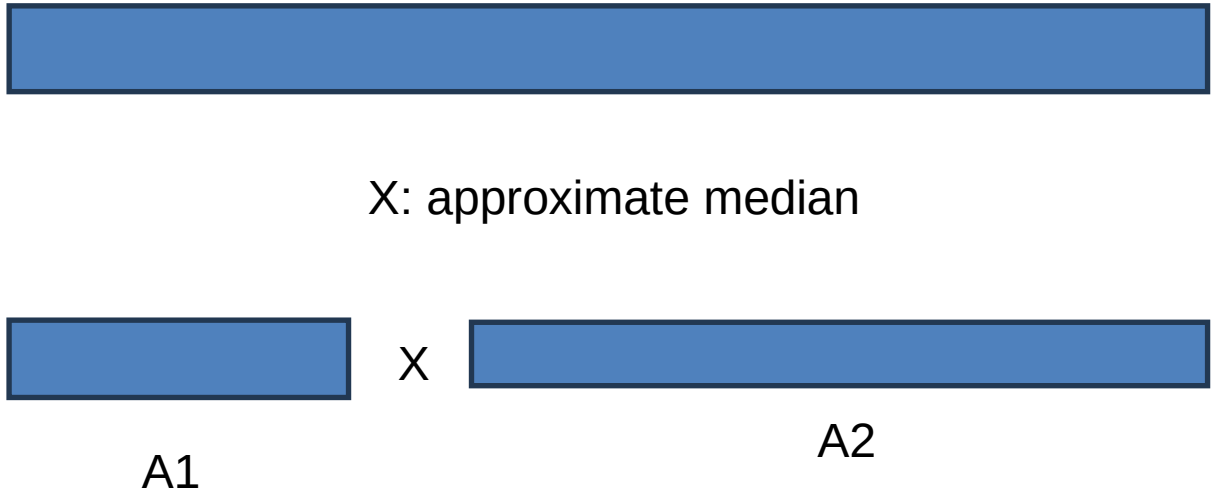




- If the rank of X is ≤ n/4, where is the median?
- n/4 ≤ |A1| and n/4 ≤ |A2| then it is the case that **|A1|, |A2| ≤ 3n/4**
- Suppose that approximate median can be found in linear time.

Element of Rank p using Approximate median

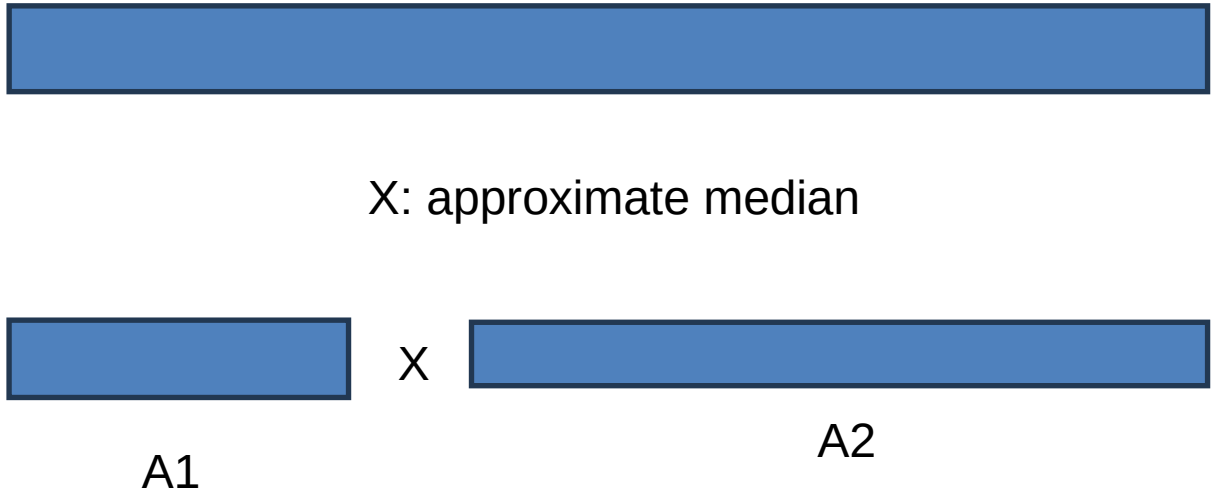
Recursion goes through!



- The median of A is the element of rank **n/2 – (|A1|+1)** in **A2**.
- Instead of looking at the median of A, we will **look for the element of rank n/2 in A**.
- Therefore in A2 we could look for the element of rank **n/2 – (|A1|+1).**

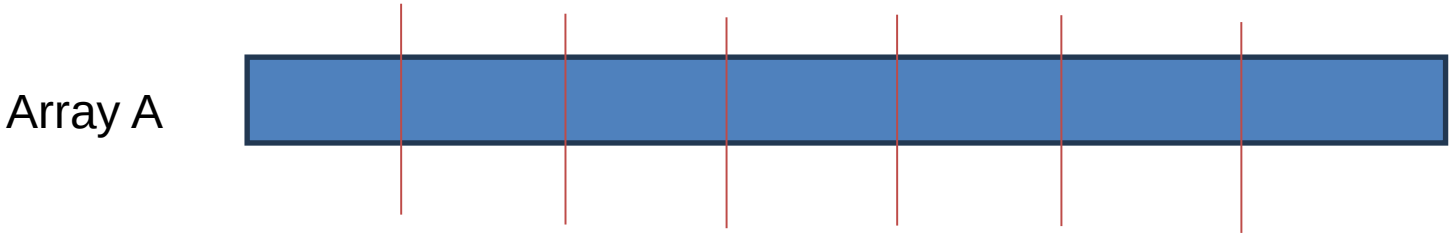
Element of Rank p using Approximate median

T(n) = c.n + T(3n/4)



Goal: Find approximate median of A in O(n) time.

Element of Rank p using Approximate median



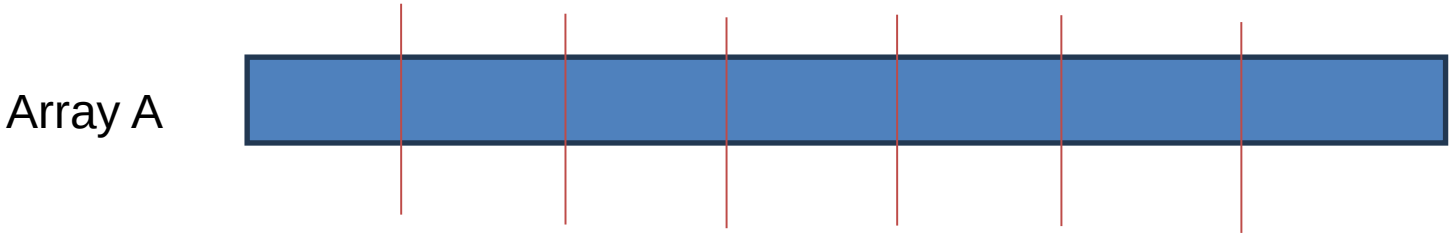
Q(A,n/2)

1. Divide the array into blocks of 5 elements. We get that #blocks = n/5.

2. Find the median of in each block, say m_i of block B_i.

3.

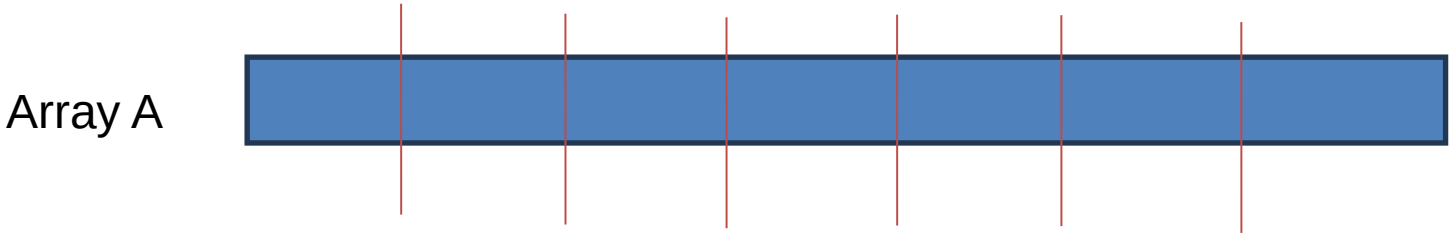
Element of Rank p using Approximate median



Q(A,n/2)

1. Divide the array into blocks of 5 elements. We get that #blocks = n/5.
2. Find the median of in each block, say m_i of block B_i.
3. Find the median X of m_1, m_2, …. , $m_{n/5}$
4. X is our approximate median for A.

Element of Rank p using Approximate median



Approximate median for Q(A,n/2)

1. Divide the array into blocks of 5 elements. We get that #blocks = n/5.
2. Find the median of in each block, say m_i of block B_i.
3. Find the median X of m_1, m_2, …. , $m_{n/5}$ **Recurse Q(M, n/10)**
4. X is our approximate median for A.

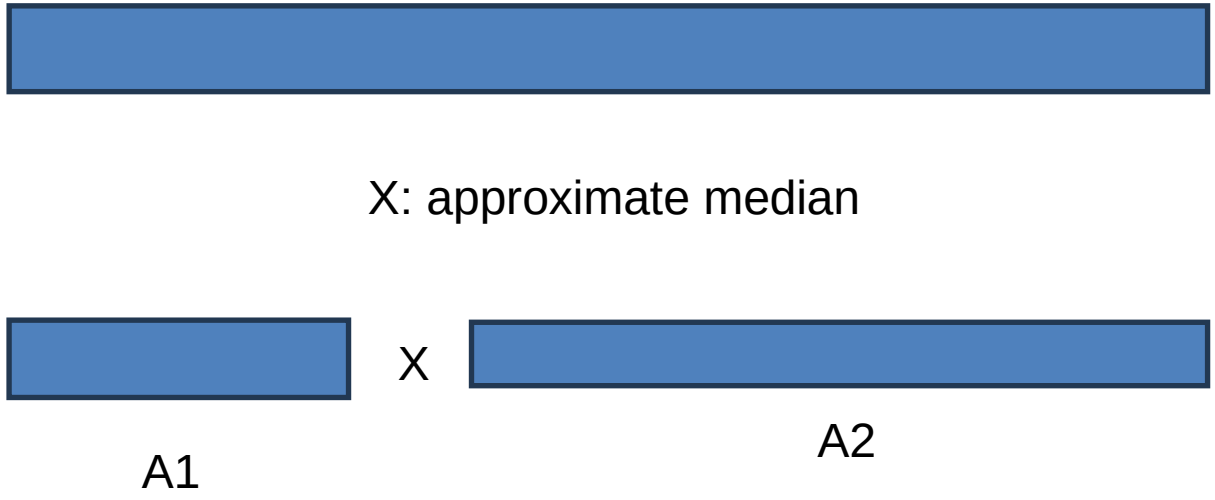
Element of Rank p using Approximate median **Q(A,n/2)**

1. Approximate Median
   - Divide the array into blocks of 5 elements. We get that #blocks = n/5.
   - Find the median of in each block, say m_i of block B_i.
   - Find the median X of m_1, m_2, …. , $m_{n/5}$ **Recurse Q(M, n/10)**

2. Partition array A based on the pivot X as per QuickSort.

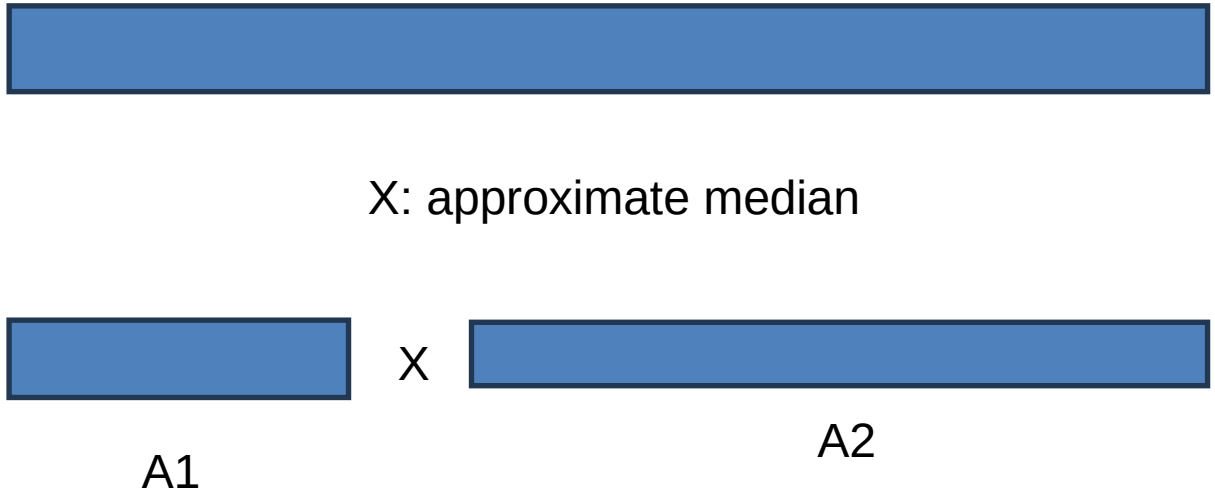
3. Recurse on the sub-problem accordingly.

Element of Rank p using Approximate median



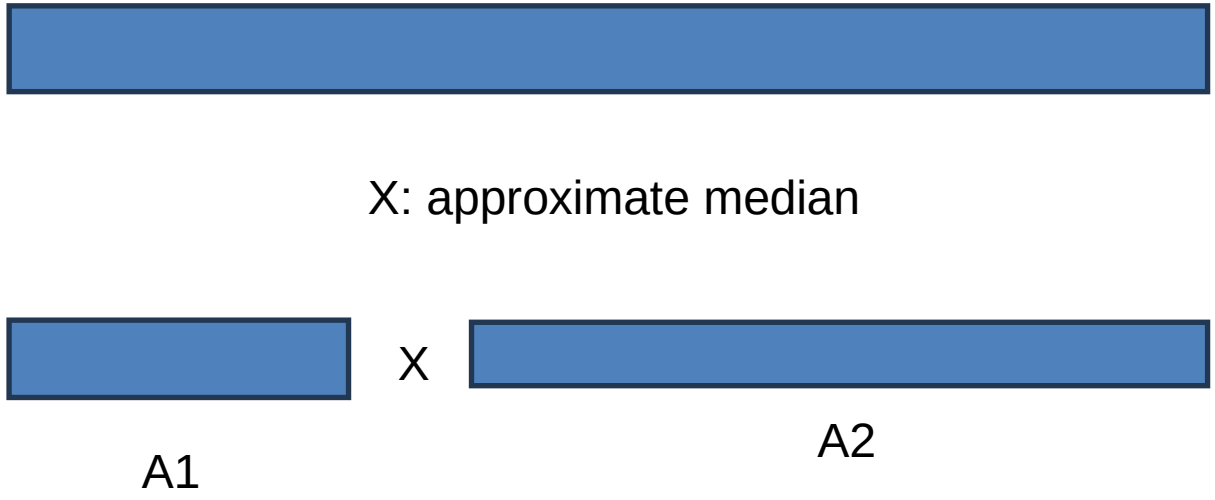
- Find an approximate median
- Recurse on the sub-problem.

Element of Rank p using Approximate median



- Find an approximate median
- Recurse on the sub-problem.
  - |A1|=p-1 then element of rank p is X
  - |A1|>p-1 then element of rank p is in A1
  - |A1|<p-1 then element of rank p is in A2

Element of Rank p using Approximate median (Source Problem Q(A, p))



- Find an approximate median
- Recurse on the sub-problem.
  - |A1|=p-1 then element of rank p is X — **Return X**
  - |A1|>p-1 then element of rank p is in A1 — **Recurse Q(A1, p)**
  - |A1|<p-1 then element of rank p is in A2 — **Recurse Q(A2, p-|A1|-1)**

Element of Rank p using Approximate median (Source Problem Q(A, p))

1. Find an approximate median

   a) Divide the array into blocks of 5 elements. We get that #blocks = n/5.

   b) Find the median of in each block, say m_i of block B_i.

   c) Find the median X of $m_1, m_2, \ldots, m_{n/5}$ **Recurse Q(M, n/10)**

2. Partition array A into A1 and A2 based on the pivot X as per QuickSort.

3. Recurse on the sub-problem.
   - |A1|=p-1 then element of rank p is X **Return X**
   - |A1|>p-1 then element of rank p is in A1 **Recurse Q(A1, p)**
   - |A1|<p-1 then element of rank p is in A2 **Recurse Q(A2, p-|A1|-1)**

Element of Rank p using Approximate median (Source Problem Q(A, p))

Running time?

1. Find an approximate median

   a) Divide the array into blocks of 5 elements. We get that #blocks = n/5.

   b) Find the median of in each block, say m_i of block B_i.

   c) Find the median X of $m_1, m_2, \ldots, m_{n/5}$ **Recurse Q(M, n/10)**

2. Partition array A into A1 and A2 based on the pivot X as per QuickSort.

3. Recurse on the sub-problem.
   - |A1|=p-1 then element of rank p is X **Return X**
   - |A1|>p-1 then element of rank p is in A1 **Recurse Q(A1, p)**
   - |A1|<p-1 then element of rank p is in A2 **Recurse Q(A2, p-|A1|-1)**

Approximate median (p=n/2)

Q(A,n/2)

Find an approximate median

- Divide the array into blocks of 5 elements. We get that #blocks = n/5.
- Find the median of in each block, say m_i of block B_i.
- Find the median X of $m_1, m_2, \ldots, m_{n/5}$ **Recurse Q(M, n/10)**

T(n) = cn + T(n/5)

Element of Rank p using Approximate median (Source Problem Q(A, p))

Running time?

2. Partition array A into A1 and A2 based on the pivot X as per QuickSort .

T(n) = cn + T(n/5) + n

Element of Rank p using Approximate median (Source Problem Q(A, p))

Running time?

2. Partition array A into A1 and A2 based on the pivot X as per QuickSort .

T(n) = cn + T(n/5) + n

3. Recurse on the sub-problem.

- |A1|=p-1 then element of rank p is X — **Return X**
- |A1|>p-1 then element of rank p is in A1 — **Recurse Q(A1, p)**
- |A1|<p-1 then element of rank p is in A2 — **Recurse Q(A2, p-|A1|-1)**

Element of Rank p using Approximate median (Source Problem Q(A, p))

Running time?

2. Partition array A into A1 and A2 based on the pivot X as per QuickSort .

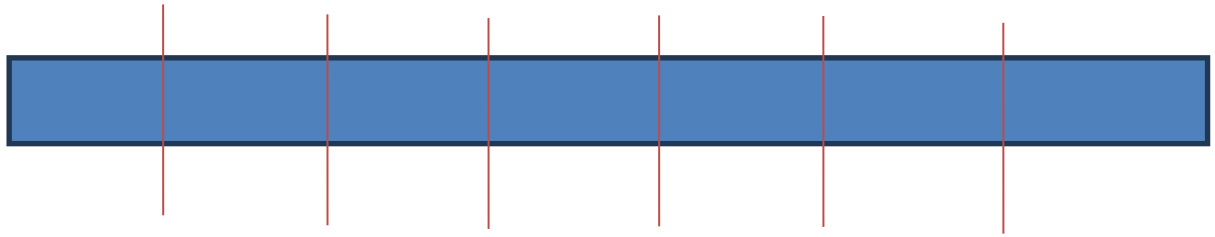
$T(n) = cn + T(n/5) + n$

3. Recurse on the sub-problem.

- |A1|=p-1 then element of rank p is X — **Return X**
- |A1|>p-1 then element of rank p is in A1 — **Recurse Q(A1, p)**
- |A1|<p-1 then element of rank p is in A2 — **Recurse Q(A2, p-|A1|-1)**

$T(n) = cn + T(n/5) + n + T(3n/4)$ → **O(n)**

Array A



B1	B2	B3	B4	B5
a15	a25	a35	a45	a55
≥	≥	≥	≥	≥
a14	a24	a34	a44	a54
≥	≥	≥	≥	≥
a13 = m1	a23 = m2	a33 = m3	a43 = m4	a53 = m5
≥	≥	≥	≥	≥
a12	a22	a32	a42	a52
≥	≥	≥	≥	≥
a11	a21	a31	a41	a51

Correctness of approximate median.

To show linearity: T(n) = O(n)

We guess that T(n) = O(n). Let $T(n) \leq dn$.

T(n) = cn + T(n/5) + n + T(3n/4)

$T(n) = (c+1)n + d(n/5) + d(3n/4) \leq dn.$

$\Rightarrow (c+1) \leq d\,(1-1/5-3/4)$

$c+1 \leq d/20.$

We need a constant d such that $d \geq 20\,(c+1)$.

To show linearity: T(n) = O(n)

We guess that T(n) = O(n). Let $T(n) \leq dn$.

T(n) = cn + T(n/5) + n + T(3n/4)

$T(n) = (c+1)n + d(n/5) + d(3n/4) \leq dn.$

$\Rightarrow (c+1) \leq d\,(1-1/5-3/4)$

$c+1 \leq d/20.$

Use induction to verify this.

We need a constant d such that $d \geq 20\,(c+1)$.